xAPI in Action

A hands-on workshop for MSc Informatics students at ISEP — send your first xAPI statement, see it land in a real LRS, and understand what makes modern learning data tick.

50–60 MIN WORKSHOP

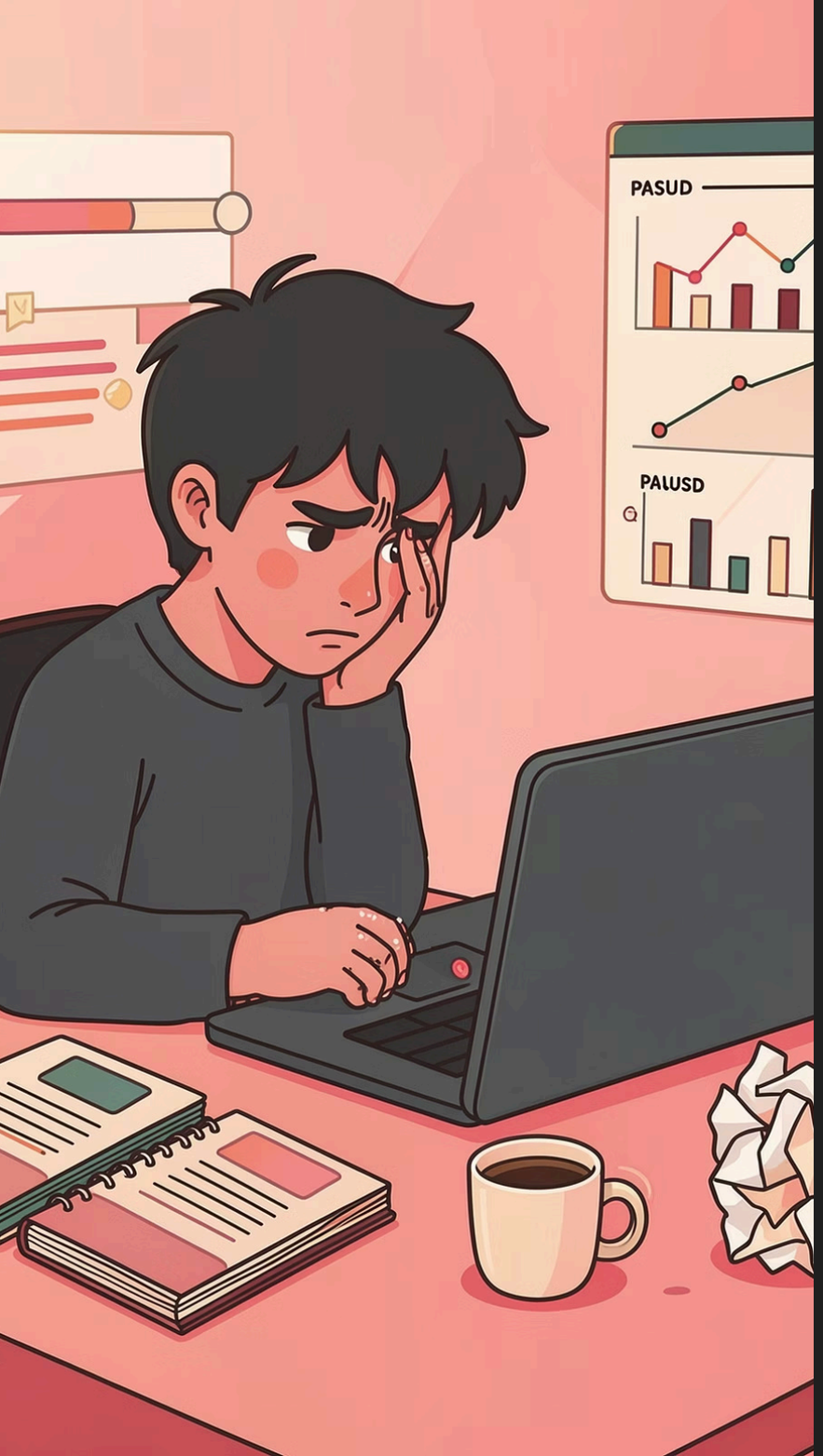
TECAA-ISEP.NETLIFY.APP

Story Beat: HOOK — We open with the problem xAPI solves.

Speaker Notes (~2 min): Welcome everyone, introduce the team, and set expectations. This is a hands-on session — you'll send real statements by the end. Open the site on your laptop now.

Mayer — Personalization: Using "you" and "your" directly addresses the learner, increasing engagement and relevance.

Made with GAMMA



The Problem: Learning Happens Everywhere – Data Doesn't Follow

📱 Mobile apps

No standard way to track in-app learning

🎮 Simulations

VR and serious games are invisible to LMS

🌐 Real world

On-the-job training leaves no digital trace

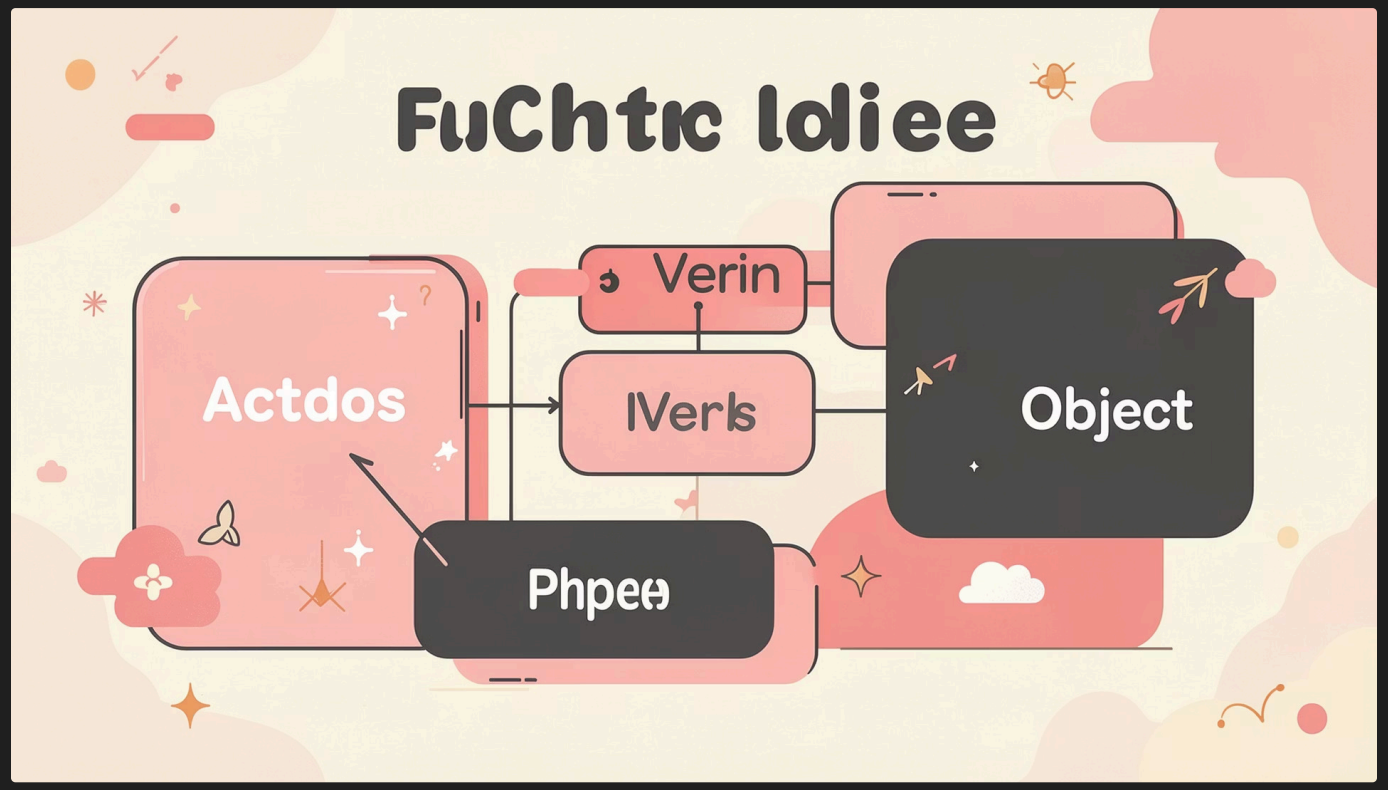
⚠ **Story Beat: BUILD** — The gap between learning and data is the problem xAPI closes.

Speaker Notes (~3 min): Ask who has used an LMS before. Most track only course completions — not what happens in apps, simulations, or on the job. That's the gap xAPI closes.

Mayer — Signaling: The three boxes visually cue the scope of the problem before introducing the solution.

Made with GAMMA

What Is xAPI? One Sentence.



xAPI is a specification that lets any learning experience send a structured **statement** — *Actor → Verb → Object* — to a Learning Record Store.

Actor

Who did it (learner, system)

Verb

What they did (completed, attempted)

Object

What they did it to (course, video)

Story Beat: BUILD — The core concept: every learning action becomes a traceable statement.

Speaker Notes (~4 min): This is the key takeaway. Every learning action — watching a video, failing a quiz — becomes a statement. The Actor–Verb–Object pattern is the heart of xAPI.

Mayer — Pre-training: Introducing Actor–Verb–Object before the demo reduces cognitive load during the hands-on task.

Made with GAMMA

The Statement Anatomy

Every valid xAPI statement must include these three fields. Everything else — context, result, timestamp — is optional but powerful.

Story Beat: BUILD — We now know the building blocks before we build anything.

Speaker Notes (~3 min): Walk through each part. Actor and Verb are always IRIs — we'll see why on the Validation page. Optional fields like *result* and *context* add depth.

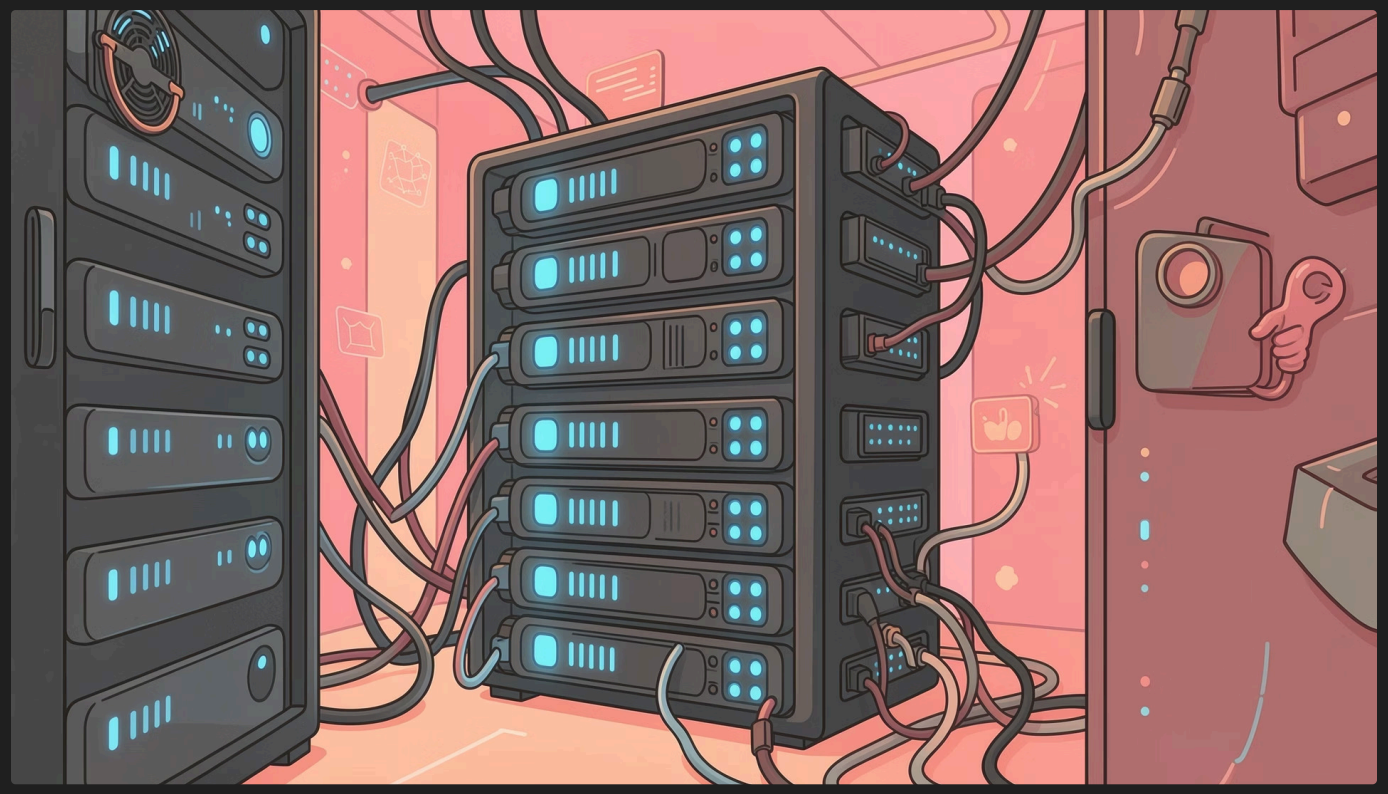
Mayer — Segmenting: Breaking the statement into three discrete parts helps learners process each component before combining them.

Made with GAMMA

The LRS: Where Statements Live

Learning Record Store

The LRS is xAPI's database — it receives, stores, and surfaces statements. Unlike an LMS, it doesn't deliver content. It just **records everything**.



Receives

HTTP POST from any client

Stores

Immutable, queryable records

Surfaces

Reports, dashboards, analytics

Story Beat: BUILD — The LRS is the destination for every statement we'll send today.

Speaker Notes (~3 min): The LRS is like a bank for learning data — it doesn't teach, it records. Our test LRS has a UI where you'll confirm your statements landed.

Mayer — Spatial Contiguity: Placing the LRS explanation next to its visual representation keeps related information together, reducing split attention.

Made with GAMMA

Our Site: Your Workshop Map

Everything you need is at **tecaa-isep.netlify.app** — bilingual, four sections, four Twine stories.

01

Fundamentals

What xAPI is, why it exists, the statement model

02

Core Structure

Actor, Verb, Object — the mandatory trio

03

Optional Fields

Context, result, timestamp — add depth

04

Validation & IRIs

Common errors, IRI rules, recovery steps

Story Beat: TURN — The site is your guide. Each section has a Twine story at `/stories/xapi-*/`.

Speaker Notes (~3 min): Navigate to the site now. Each section has a companion Twine story — your instructor will assign which one you start with. The site is bilingual (EN/PT).

Mayer — Coherence: Only the four relevant sections are shown — no extra navigation options or unrelated content that could distract.

Made with GAMMA

Live Demo: Sending Your First Statement

The proxy is a Netlify serverless function — your API credentials stay hidden in environment variables, never in the browser.

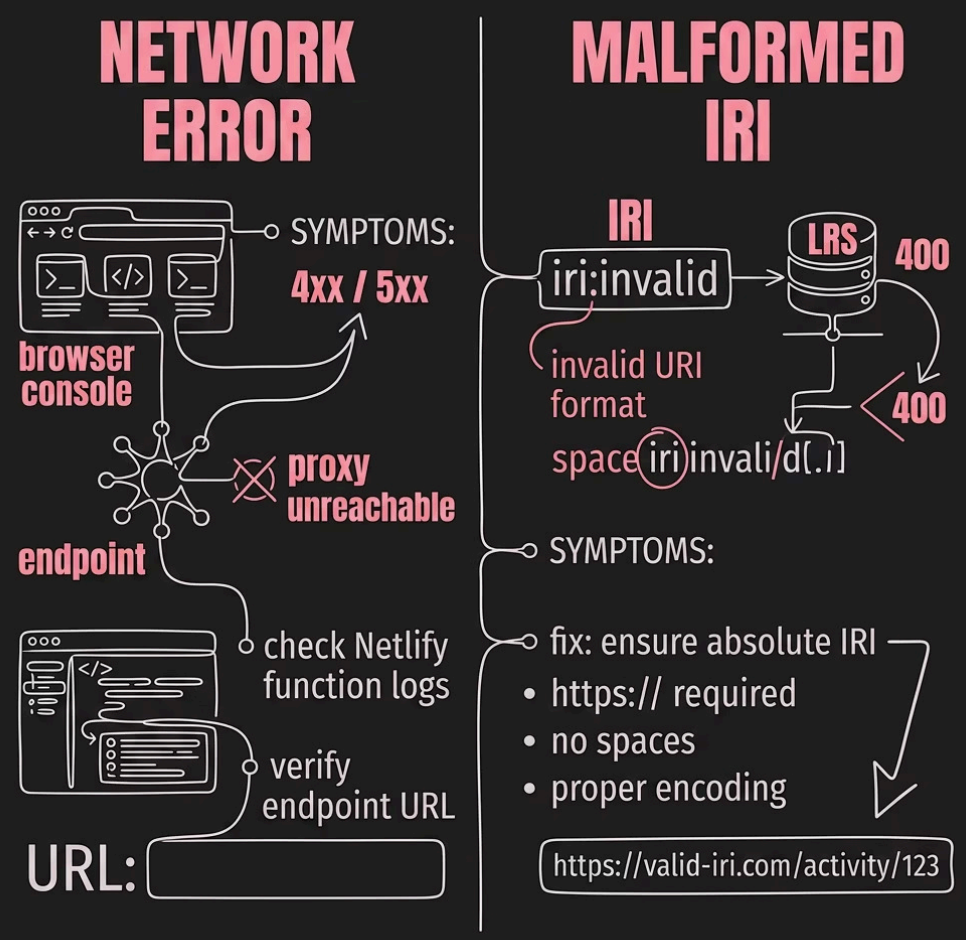
⚠ **Story Beat: PAYOFF** — This is the moment: you send a real statement to a real LRS.

Speaker Notes (~5 min demo + 15 min hands-on): Walk through each step live. Then give participants 15 minutes to try it themselves. Circulate and help anyone stuck.

Mayer — Temporal Contiguity: Showing the steps immediately before participants perform them keeps explanation and action synchronized.

Made with GAMMA

When Things Go Wrong



Debug Like a Pro

- Open DevTools → Network tab first
- Check the proxy function logs on Netlify
- Validate IRIs with an online checker
- Re-read the Validation & IRIs section

> The Validation page on our site covers both errors with examples.

Story Beat: PAYOFF — Errors are expected. Knowing how to recover is part of the skill.

Speaker Notes (~5 min): These two errors cover 90% of issues. Show the DevTools Network tab live. Encourage participants to use the Validation section as a reference.

Mayer — Modality: Pairing the visual infographic with spoken explanation (not on-screen text) uses both channels without overloading either.

Made with GAMMA

Your Turn: Workshop Tasks

Task 1 – Baseline

Send a minimal Actor–Verb–Object statement. Confirm it appears in the LRS.

Task 2 – Enrich

Add *result* (score) and *context* (grouping). Re-send and compare.

Task 3 – Break & Fix

Intentionally break an IRI. Trigger the error. Recover using the Validation page.

Story Beat: PAYOFF — Three tasks, increasing complexity. You'll leave having done all three.

Speaker Notes (~15 min): Participants work through tasks in order. Task 3 is the hardest — encourage pairing up. Check in at the 8-minute mark.

Mayer — Voice: Direct, conversational task descriptions ("Break & Fix") feel like a peer talking, not a textbook, which boosts motivation.

Made with GAMMA

Wrap-Up & What's Out of Scope

✅ You Can Now

- Define xAPI and the Actor–Verb–Object model
- Send a statement via our proxy to the test LRS
- Diagnose network errors and malformed IRIs
- Navigate all four site sections and Twine stories

⚠️ Know Before You Leave

- **Confusion risk:** IRIs vs. URLs — the site clarifies this
- **Confusion risk:** Proxy feels like magic — check Netlify logs
- **Confusion risk:** Twine story order — follow your instructor's assignment
- **Verify before May 12:** Test LRS is live and accepting statements
- **Verify before May 12:** Netlify proxy env vars are correctly set

⚠️ **Out of scope:** xAPI profiles, conformance requirements, and production LRS setup.

Story Beat: CODA — You came with questions. You leave with a statement in the LRS.

Speaker Notes (~5 min): Recap the five learning objectives. Point to the site for further study. Collect feedback forms. Remind everyone the site stays up after the workshop.

Mayer — Redundancy: Summarizing key takeaways verbally while showing them on-screen reinforces retention without introducing new information.

Made with GAMMA